

Dynamique des connaissances - Projet

Par Artus Bleton, Guilhem Dupuy, Anh Duy Vu

Le lien vers l'application : <https://inf2347m.streamlit.app/>

Note : Il se peut que vous deviez attendre **2 à 3 minutes** pour que Streamlit initialise l'application.

Pour la **Partie 2**, le chargement peut prendre environ **1 minute** car le modèle est téléchargé depuis Hugging Face.

Partie 1 : Reasoning with ABA

1.1. Introduction

Ce rapport documente la Partie 1 du CC2, qui porte sur la mise en œuvre d'un système complet pour les cadres d'argumentation basée sur les Hypothèses (ABA) prenant en charge les extensions basées sur les préférences (ABA+). Le système inclut le parsing du cadre, la génération des arguments, le calcul des attaques, ainsi qu'une interface web interactive. Même si la mise en œuvre technique a été un succès, l'aspect le plus précieux de cette première partie a été le processus de découverte et de correction d'une incompréhension fondamentale concernant le fonctionnement des sémantiques d'ABA+. Ce rapport se concentre sur la structure du projet, les défis conceptuels rencontrés lors du développement, et les leçons tirées de cette expérience.

1.2. Vue d'ensemble du système et approche

Cette première partie du projet met en œuvre les fonctionnalités principales des cadres ABA/ABA+.

Le système est composé de deux fichiers Python principaux, chacun ayant un rôle distinct :

- **app.py** contient le moteur principal, avec toute la logique d'argumentation. Il définit notamment la classe `ABAFramework`, responsable du parsing des définitions textuelles de cadres, la classe `PreferenceOrder` pour la gestion des relations de préférence, les procédures de normalisation (élimination des cycles, aplatissement et atomisation), l'algorithme de dérivation directe pour la génération d'arguments, ainsi que les fonctions de calcul d'attaques pour les sémantiques ABA et ABA+. Ce fichier représente la base théorique du système et peut être utilisé indépendamment comme bibliothèque.
- **streamlit_app.py** fournit la couche d'interface utilisateur. Il importe les fonctionnalités principales depuis **app.py** et les intègre dans une application web interactive construite avec Streamlit. Ce fichier gère la saisie utilisateur (textes, cases

à cocher), capture les sorties du moteur principal et affiche les résultats dans un panneau interactif avec possibilité de téléchargement. L'interface a été conçue pour être simple et claire, avec toutes les fonctionnalités réunies sur une seule page et une documentation intégrée.

L'application a été déployée sur Streamlit Cloud, une plateforme gratuite permettant d'héberger directement des applications Streamlit à partir de dépôts GitHub. Le processus de déploiement a été simple : nous avons poussé le code sur GitHub avec un fichier `requirements.txt` (spécifiant `streamlit` et `plotly`), puis connecté le dépôt à Streamlit Cloud. La plateforme détecte automatiquement le fichier `streamlit_app.py` comme point d'entrée et gère le déploiement, rendant l'application accessible via une [URL publique](#). Cette solution permet un partage facile et une expérimentation sans installation locale de Python ni de dépendances.

Le parseur accepte une syntaxe flexible incluant les déclarations de langage, les hypothèses, les correspondances de contraires, les règles d'inférence et les relations de préférence. Trois procédures de normalisation optionnelles sont disponibles :

- **Élimination des cycles** : supprime les règles circulaires.
- **Aplatissement** : empêche les hypothèses d'apparaître en tête de la règle.
- **Atomisation** : remplace les non-hypothèses dans les corps de règle par de nouvelles hypothèses réfutables.

Ces transformations préservent la sémantique tout en simplifiant la structure du cadre. Les arguments sont générés via la dérivation directe, qui construit systématiquement les conclusions à partir des hypothèses et des règles, tout en maintenant la minimalité des supports.

Pour des exemples typiques (3 à 5 hypothèses, 5 à 10 règles), la génération se fait en quelques millisecondes. Le système gère efficacement jusqu'à une quinzaine d'hypothèses, ce qui est suffisant pour un usage pédagogique.

1.3. Le défi central : mauvaise compréhension des sémantiques d'attaque ABA+

Au départ, nous avons commis une erreur conceptuelle majeure. Ayant correctement implémenté l'ABA standard (où les arguments attaquent d'autres arguments), nous avons cru qu'ABA+ se limitait à ajouter une vérification de préférence. Nous pensions donc que si un argument A attaquait B dans ABA, il suffirait de vérifier si A était moins préféré que B dans ABA+.

Cependant, les résultats obtenus sur un exemple simple ne correspondaient pas à la littérature : le comptage des attaques normales et inversées était incorrect, et certaines classifications d'attaques semblaient incohérentes. Après de nombreuses vérifications, nous avons compris que le problème venait de notre conception même. En réalité, ABA+ ne définit pas les attaques entre arguments individuels, mais entre ensembles d'hypothèses.

Cette distinction est essentielle et change complètement la manière de calculer les attaques. De plus, ABA+ introduit une symétrie absente dans ABA.

Un ensemble X peut attaquer un ensemble Y de deux façons :

- **Attaque normale** : X dérive le contraire d'une hypothèse de Y sans désavantage de préférence.
- **Attaque inversée** : Y dérive le contraire d'une hypothèse de X, mais avec un avantage de préférence qui inverse la direction de l'attaque.

Autrement dit, il faut vérifier les deux directions pour chaque paire d'ensembles d'hypothèses. Par exemple, dans un cadre avec :

- **Hypothèses**: a, b
- **Contraires**: d, c
- **Règles**: $d \leftarrow b, c \leftarrow a$
- **Préférence**: $a < b$

L'approche basée sur les arguments produit des résultats erronés. Avec l'approche basée sur les ensembles, on examine :

- si l'ensemble $\{a\}$ attaque $\{b\}$,
- puis si $\{b\}$ attaque $\{a\}$, en considérant les préférences dans les deux sens.

Cette redéfinition nous a obligés à redéfinir entièrement le calcul des attaques, en énumérant tous les sous-ensembles non vides d'hypothèses et en vérifiant les deux types d'attaques pour chaque paire.

1.4. Défis supplémentaires et solutions

D'autres difficultés pratiques sont apparues. Par exemple, la syntaxe des préférences : certains articles utilisent $a > b$ (a préféré à b), d'autres $a < b$ (a moins préféré que b). Notre parseur initial ne reconnaissait que $>$, ignorant silencieusement les préférences écrites avec $<$, ce qui faisait involontairement retomber ABA+ sur ABA standard. Nous avons corrigé cela en acceptant les deux notations et en convertissant tout en représentation interne cohérente.

Un autre problème concerne l'affichage des attaques : celles-ci sont représentées par des triplets (X, Y, β) , où X attaque Y via l'hypothèse β . Or, plusieurs attaques pouvaient concerner les mêmes ensembles X, YX, YX, Y, ce qui gonflait artificiellement le nombre total d'attaques affichées. Nous avons résolu ce problème en regroupant les attaques par paires (X, Y) et en listant toutes les hypothèses cibles ensemble, donnant une représentation plus claire et fidèle.

1.5. Résultats et leçons tirées

Nous avons validé le système final sur un cadre avec trois hypothèses a, b, c et huit arguments (faits, hypothèses atomiques et conclusions dérivées).

Le système a correctement identifié :

- 10 attaques ABA standard,
- 12 attaques normales et 8 attaques inversées pour ABA+ avec la préférence $b < a$

La validation s'est faite par : calcul manuel sur de petits sous-cadres, vérification de la cohérence ABA+ (sans préférences) = ABA, contrôle de la minimalité des supports, et comparaison avec les exemples du cours et des TD.

Les tests unitaires minimaux (cadres simples à 2 hypothèses) ont été essentiels : ils ont permis de repérer rapidement les erreurs conceptuelles. Nous avons appris qu'il faut toujours construire des cas de test simples et ciblés avant de tester des exemples complexes.

Enfin, la présentation des résultats a évolué : d'abord un texte brut, puis des tableaux structurés, et enfin des visualisations interactives (PyVis). Offrir plusieurs formats de sortie s'est révélé plus efficace : les chercheurs préfèrent les tableaux précis, tandis que les apprenants bénéficient des graphes visuels.

1.6. Conclusion

La Partie 1 du projet a livré un système complet ABA/ABA+ avec une implémentation correcte des sémantiques d'attaque standard et basées sur les préférences. L'architecture en deux fichiers sépare efficacement la logique et l'interface, et le déploiement sur Streamlit Cloud rend le système facilement accessible.

Avec plus de temps, les extensions possibles incluraient : le calcul des extensions admissibles, préférées, stables, fondées et idéales, la possibilité de tracer en détail les dérivations d'arguments, la comparaison côte à côte entre ABA et ABA+. Enfin, une compatibilité avec les formats standards d'argumentation permettrait une interopérabilité accrue avec d'autres outils.

Partie 2 : Relation-based argument mining

Le modèle de détection des attaques entre arguments utilisé dans cette partie est disponible en ligne : <https://huggingface.co/duybeobn1/my-finetuned-model>.

Ce dépôt contient les poids du modèle fine-tuné, la configuration ainsi que les fichiers du tokenizer nécessaires au bon fonctionnement. Le modèle est chargé directement depuis cette adresse lors de l'exécution de l'application, ce qui facilite sa mise à jour et son déploiement sans nécessiter de stockage local volumineux.

2.1. Introduction

Cette seconde partie s'intéresse à l'analyse automatique des relations entre arguments, dans la continuité du champ du *Relation-Based Argument Mining*.

L'objectif est de concevoir un modèle capable d'identifier, à partir de deux arguments textuels, si le second **soutient** ou **attaque** le premier.

Pour cela, j'ai constitué un corpus à partir de la plateforme de débats **Kialo**, puis entraîné différents modèles de langage afin d'évaluer leur capacité à reconnaître ces relations argumentatives complexes.

2.2. Construction du jeu de données

J'ai choisi de ne traiter que les **paires enfant–parent** dans les discussions de Kialo, plutôt que d'autres types de relations (par exemple frère–frère ou croisées).

En effet, les liens enfant-parent sont clairement définis dans la hiérarchie de Kialo, alors que d'autres relations peuvent être ambiguës, même pour un humain.

À partir des **250 discussions les plus actives et mieux classées** du site, j'ai extrait toutes les paires (*argument enfant* → *argument parent*) avec le label suivant :

- **0 = Support** : l'argument enfant soutient son parent.
- **1 = Attack** : l'argument enfant s'oppose à son parent.

Le découpage des données a été réalisé **par discussion** (70 % / 15 % / 15 %) afin d'éviter que des arguments d'une même discussion apparaissent dans plusieurs sous-ensembles (entraînement, validation, test).

2.3. Exploration et nettoyage des données

Un script de nettoyage a été appliqué pour standardiser le corpus :

- normalisation Unicode (NFKC) ;
- suppression des URL, balises HTML et références numériques ;
- suppression des doublons ;
- filtrage des arguments trop courts ;
- troncature à **512 tokens maximum**.

L'exploration a montré une **distribution équilibrée** entre les classes *support* et *attaque* après sous-échantillonnage.

Split	Nombre d'exemples (n)	Équilibre des labels (0 / 1)	Longueur moyenne (text_a)	Longueur moyenne (text_b)	Doublons retirés
Train	149 166	74 802 / 74 364	24,0 mots (p95 = 49)	21,6 mots (p95 = 45)	590
Validation	25 427	12 649 / 12 778	23,0 mots (p95 = 46)	20,5 mots (p95 = 42)	48
Test	30 246	15 581 / 14 665	23,3 mots (p95 = 45)	20,4 mots (p95 = 40)	7

2.4. Modèles et protocole d'apprentissage

Deux modèles ont été fine-tunés sur le même jeu de données nettoyé et équilibré, constitué de paires d'arguments enfant-parent.

La tokenisation a été effectuée par paire (*text_a*, *text_b*) avec troncature à une longueur maximale comprise entre 128 et 256 tokens selon le modèle.

L'apprentissage a été mené avec la librairie Hugging Face Transformers via la classe Trainer, selon les mêmes paramètres généraux :

- **Optimiseur** : AdamW avec un *learning rate* de 2×10^{-5} et une décroissance linéaire (*scheduler*) après un *warmup* de 10 %.
- **Régularisation** : *weight decay* = 0,01.
- **Batches** : 8 en entraînement, 16 en validation.
- **Critère de sélection** : meilleur modèle retenu selon la métrique **F1** après chaque époque.

a) BERT-Tiny

Version minimale de BERT, comportant 2 couches d'attention, une taille cachée de 128, et environ 4 millions de paramètres.

Ce modèle léger a été choisi pour un premier entraînement rapide sur CPU, sans calcul en précision mixte.

L'apprentissage a été réalisé sur 10 époques, avec une longueur maximale de 256 tokens par paire, afin d'obtenir une première estimation des capacités du modèle sur la tâche.

Métrique	Classe 0 (Support)	Classe 1 (Attack)	Moyenne / Globale
Précision	0,698	0,611	0,655
Rappel	0,554	0,746	0,650
F1-score	0,618	0,672	0,645
Exactitude globale			0,647

b) DeBERTa-v3-base

Modèle plus avancé de Microsoft, doté de 12 couches, d'une taille cachée de 768, et d'environ 86 millions de paramètres.

Il introduit une attention désentrelacée (Disentangled Attention) et une modélisation améliorée des positions relatives, ce qui renforce sa compréhension du contexte.

L'entraînement a été effectué sur GPU, avec calcul en fp16, pendant 5 époques, et une longueur maximale de 128 tokens par paire.

Ce modèle a servi de référence principale pour la classification des relations argumentatives.

Métrique	Classe 0 (Support)	Classe 1 (Attack)	Global
Précision	0.860	0.903	
Rappel	0.915	0.842	
F1-score	0.887	0.872	0.880
Exactitude globale	–	–	0.880

	Prédit : Support	Prédit : Attack
Réel : Support (0)	14 261	1 320
Réel : Attack (1)	2 316	12 349

2.5. Discussion

Le modèle **DeBERTa-v3-base** surpasse largement BERT-tiny, atteignant un score F1 moyen de **0,88**, comparable aux meilleurs résultats de la littérature pour la fouille de relations argumentatives (souvent $F1 \approx 0,85\text{--}0,90$ sur des corpus de référence tels que *UKP ArgMining* ou *IBM ArgQ*).

Les erreurs résiduelles concernent surtout des **cas de nuance sémantique** : par exemple, un argument qui semble s'opposer mais exprime en réalité un accord partiel ou conditionnel. Ces limites reflètent la difficulté de modéliser la **polarity implicite** dans le raisonnement argumentatif.

Partie 3 - Visualizing acceptability degree spaces for gradual semantics

Cette partie vise à explorer la représentation géométrique des états d'acceptabilité d'un système d'argumentation, à travers la visualisation de l'espace des poids et de leur convergence pour la sémantique weighted h categorizer.

3.1 - Argumentation framework

La première étape de cette partie a été de créer une modélisation simple de système d'argumentation, afin de pouvoir facilement en créer et en manipuler dans la suite de cette partie. La classe créée s'appelle **AFMatrix**, contenue dans le fichier dédié suivant : [argumentationFramework.py](#).

Description du code :

Ce code ne contient rien de complexe ni de particulièrement intéressant. Il modélise simplement un argumentation framework en tant qu'objet python, avec les 3 attributs conformes à la modélisation communément utilisée :

- A : ensemble des arguments du framework
- R : matrice des attaques
- w : vecteur des poids associés aux arguments de A

Liste des fonctions de l'objet **AFMatrix** :

- **set_attack_manual(self, attacker, attacked)** : définit manuellement une relation d'attaque entre deux arguments donnés, en mettant la valeur correspondante à 1 dans la matrice R.
- **set_attack_matrix(self, Rmatrix)** : remplace la matrice des attaques par une matrice R complète fournie (sous forme de numpy.ndarray ou pandas.DataFrame), après vérification de sa validité.
set_weight_manual(self, arg, weight) : assigne manuellement un poids donné à un argument spécifique dans le vecteur des poids w.
set_weight_matrix(self, w) : définit l'ensemble des poids des arguments à partir d'un tableau numpy de taille appropriée, avec vérification du type et de la forme.
- **__str__(self)** : renvoie une représentation textuelle lisible de l'objet AFMatrix, affichant la liste des arguments, la matrice d'attaques et les poids associés.
- **generate_random_AF(n, weight_range=(0.0, 1.0), seed=None)** : génère automatiquement un framework d'argumentation aléatoire de taille n, avec une matrice d'attaques aléatoires et des poids compris dans l'intervalle spécifié.

3.2 - Weighted H Categorizer

La seconde étape a été d'implémenter un algorithme calculant cette sémantique. Les particularités de cette sémantique :

- se base sur les poids associés à chaque argument
- récursive : se calcule par itérations, la valeur $HC(k+1)$ étant basée sur $HC(k)$
- convergence : par conséquent, itérations répétées jusqu'à convergence

La problématique principale de l'implémentation : être efficace. Dans la suite du projet, il faudra exécuter le calcul de weighted H categorizer 100 000 fois. Il faut donc un code le plus optimisé possible. Cet algorithme est codé dans la fonction `weighted_h_categorizer` du fichier `weightedHCategorizer.py` dédié. Les paramètres de cette fonction sont :

- un objet de type `AFMatrix` représentant un argumentation framework
- `epsilon` : paramètre définissant la convergence. Si aucun poids n'est ajusté de plus d'epsilon lors d'une itération, on considère qu'il y a convergence
- `max_iter` : nombre limite d'itérations.

`Max_iter` a été ajouté comme sécurité, afin d'éviter que le programme ne tourne trop longtemps lors de l'appel de cette fonction 100 000 fois. Si `max_iter` est atteint avant convergence, nous renvoyons le résultat et écrivons un log avertissant de la non convergence.

3.3 - Génération du convex-Hull de points obtenus et visualisation

Objectif :

Le but de cette partie est dans un premier temps de générer un ensemble aléatoire de 100 000 vecteurs w de taille n . Ils représentent 100 000 exemples potentiels de poids d'un argumentation framework de taille n , normalisés dans $[0 ; 1]$.

En prenant 100 000 ensembles pondérés, on peut considérer que nous avons une approximation de l'ensemble des sets de poids existants, capturant ainsi tous les équilibres / déséquilibres que nous pourrions potentiellement rencontrer dans un argumentation framework. La normalisation simplifie les calculs pour la suite, mais les relations entre les poids sont conservées : on ne perd pas d'information.

Ensuite, nous appliquons ces poids à un framework donné (qui sera donné en entrée dans notre application), et faisons converger chacun de ces sets vers une valeur finale (également dans $[0 ; 1]$) grâce à l'algorithme `weighted_h_categorizer` décrit précédemment. Il nous renvoie en d'autres termes un ensemble de 100 000 points dans un espace de dimension n (nombre d'arguments du framework choisi).

Nous approximations ainsi l'ensemble des possibles valeurs des poids du framework, après leur convergence par l'algorithme `weighted_h_categorizer`.

Finalement, nous calculons l'enveloppe convexe de cet ensemble de points. Cette enveloppe est calculée à partir d'un sous ensemble de points limites, appelés "sommets" du hull, elle a pour propriété d'envelopper l'ensemble de nos points par construction de plans - les "faces" du hull - qui relient ensemble les "sommets". Cette étape finale permettra une visualisation propre du nuage de points obtenu.

Problèmes rencontrés

La première difficulté, mentionnée précédemment, est l'efficacité. Il faut générer 100 000 vecteurs, et les faire converger via notre algorithme `weighted_h_categorizer`. Pour gagner en efficacité, nous avons privilégié l'usage de float32 VS 64, ainsi que la génération des 100000 vecteurs aléatoires d'un seul coup dans la fonction `weightedHCategorizerGenerator`. Bien que ce soit la phase la plus lente de notre code, il est capable de résoudre cette étape en un temps variant entre 30 secondes (Framework de taille $n = 1$ ou 2) et 2 minutes ($n=18$). C'est sans doute l'un des éléments de ce projet qui mériterait d'être amélioré, mais en l'état nous avons considéré ce résultat comme acceptable.

La seconde, plus problématique, est l'explosion du temps de calcul d'un convex Hull avec le nombre de dimensions :

- $n = 5$, temps ≤ 0.02 sec
- $n = 6$, temps > 20 sec
- $n = 7$, temps > 120 sec
- $n \geq 8$, temps non raisonnable pour nos GPU

Avec n le nombre d'arguments du framework utilisé.

Choix numéro 1 : calcul du convex hull sur 3 dimensions uniquement

Après une réflexion plus approfondie, nous sommes arrivés au questionnement suivant : la représentation visuelle du convex hull devra se faire au maximum en 3 dimensions pour être interprétable à nos yeux. Donc calculer le convex hull sur plus de 3 dimensions a-t-il bien un intérêt ? En d'autres termes, perdons-nous de l'information en le calculant sur 3 dimensions uniquement, ce qui rendrait le calcul trivial pour l'ordinateur ?

Pour répondre à cette question, nous avons donc analysé ce que signifiait représenter un convex hull en 3D à partir d'un nuage de points en n D.

Nous pourrions calculer le convex hull en n dimensions, et afficher uniquement sa projection sur les 3D qui nous intéressent. C'est une méthode qui fonctionnerait, moyennant certes un long temps de calcul selon la valeur de n . Cependant, cette méthode ne permettrait pas d'affichage dynamique : la visualisation obtenue serait unique à chaque framework, et n'est pas compatible avec l'ajout de sliders ajustables par un utilisateur qui représentent les $n-3$ dimensions non affichées.

Nous avons donc conclu que cette approche du calcul de hull sur les 3 dimensions affichées uniquement était non seulement valide, mais également la seule approche valide pour un affichage dynamique.

Choix numéro 2 : le calcul des hyper-slices

Une fois la décision prise de calculer le convex hull sur uniquement les 3 dimensions visualisées, s'est posée la question suivante : comment le calculer de manière dynamique, de sorte à avoir une visualisation évolutive en fonction de la valeur des sliders ?

Pour le calculer, nous avons besoin de points. Une première option est de prendre l'ensemble de points ayant pour chacune des dimensions non représentées la valeur

indiquée sur le slider correspondant, et de calculer le convex hull de cet ensemble. Cependant, ce nombre de points est réduit, surtout en dimensionnalité élevée.

La solution : sélectionner des hyper-slices de points, ayant des valeurs dans un intervalle de tolérance autour des valeurs indiquées par les sliders. La nouvelle problématique est alors : comment définir cet intervalle de tolérance pour avoir suffisamment de points pour générer un convex hull satisfaisant, même en dimensionnalité élevée ? Le tout sans sélectionner un intervalle trop grand qui aurait peu de sens

Nos solutions retenues pour cette méthode :

- Intervalle de tolérance = $\text{ratio_souhaité puissance } (1/(n_dim - 3))$. Cette méthode de calcul a l'avantage de s'adapter au nombre de dimensions choisies. Dans le cas d'une distribution uniforme des valeurs des points sur chaque dimension, il renvoie un nombre de points égal à $\text{ratio_souhaité} * \text{nb_points_initiaux}$.
- Quand les sliders sont trop proches de 0 ou de 1, et que l'intervalle de confiance "déborde" hors des bornes [0 ; 1], nous avons opté pour une ré-attribution circulaire des valeurs (-0.05 => 0.095), jugée plus élégante. Elle permet une fluidité d'affichage du modèle, car elle évite que le convex hull affiché reste inchangé lorsqu'on bouge un slider proche de 0 ou de 1.

Remarques :

D'après nos tests, la distribution des points n'est pas uniforme : nous avons parfois jusqu'à moitié moins de points que prévu (10 000 visés, 5 500 obtenus). Une explication possible est que la répartition des points ne soit pas uniforme à cause d'une sur-représentation des valeurs extrêmes (0 et 1), liées au fait qu'elles sont calculées par convergence qui intuitivement peuvent avoir tendance à plus faire ressortir les valeurs extrêmes.

Limites de cette méthode :

Plus le nombre de dimensions augmente, plus l'intervalle de tolérance s'élargit. C'est une limitation de ce calcul d'hyper-slices : pour $n=10$ dimensions, pour conserver 10% de nos points pour la représentation visuelle, nous devons viser un intervalle de tolérance de 0.72. Si un slider indique 0.4 pour une dimension X, on sélectionne tous les points dont la valeur est sur l'intervalle [0.04 ; 0.76].

On voit donc qu'en augmentant le nombre de dimensions, la représentation perd vite en précision, même en réduisant drastiquement le nombre de points souhaités. Ainsi, garder 1% des points au lieu de 10% ne mène qu'à un faible gain : l'intervalle de tolérance passe de 0.72 à 0.52, toujours pour $n=10$ dimensions.

Description du code :

Le code de cette partie est contenu dans le fichier `convexHullGenerator.py`. Il est composé de 3 fonctions :

- `weightedHCategoryGenerator(A, R, progress_callback = None)` : prend en entrée les matrices A et R de notre framework. Le paramètre `progress_callback` permet de suivre de l'avancée de l'exécution de cette méthode depuis notre application. Cette méthode génère aléatoirement 100 000 sets de poids, puis les fait passer par l'algorithme de `weighted_h_categorizer` avant de les renvoyer.

- `hullSliderTolerance(n_dim, n_points_initiaux, n_points_min)` : méthode de calcul de l'intervalle de confiance des sliders, en fonction du nombre de dimensions n , et du ratio entre le nombre de point minimaux souhaité (`n_points_min`) et le nombre de points initiaux (`n_points_initiaux = 100 000`). Renvoie l'intervalle de confiance.
- `filter_and_project(Points, chosenDims, sliders, tolerance)` : à partir des 100 000 points générés (`Points`), des dimensions choisies (liste `chosenDims`), des valeurs des sliders, de l'intervalle de tolérance, renvoie les points de l'hyper-slice sélectionnée. C'est dans cette méthode qu'est appliqué l'intervalle de tolérance circulaire.

3.4 - Application pour la visualisation interactive des résultats

L'objectif de cette dernière partie du projet est de fournir une interface graphique interactive permettant de visualiser les ensembles de points générés et les enveloppes convexes (convex hulls) correspondantes.

Cette interface doit permettre :

- de définir un cadre d'argumentation (manuellement ou automatiquement)
- de lancer la génération des points par la méthode `weightedHCCategorizerGenerator`
- de choisir dynamiquement les dimensions à visualiser
- d'ajuster les valeurs des dimensions non représentées, grâce à l'ajout de sliders
- et enfin de visualiser en temps réel le convex hull correspondant à la configuration choisie.

Cette application constitue ainsi la jonction entre le calcul théorique (parties précédentes) et l'exploration visuelle interactive des résultats.

La théorie ayant déjà été détaillée dans les parties précédentes, nous allons principalement ici détailler le code, puis nos observations et remarques suite aux tests menés.

Choix techniques et structure générale

Le code de l'application est contenu dans le fichier `app_streamlit.py`. L'implémentation repose sur le framework Streamlit, choisi pour sa simplicité de déploiement et sa capacité à construire rapidement une interface web interactive en Python.

L'app suit une logique linéaire assez simple, sans complexité particulière. Elle est découpée en blocs successifs, correspondant aux étapes de l'expérience utilisateur. La seule complexité réside dans le filtrage dynamique et la visualisation temps réel du convex hull projeté, qui restent très réactifs dans la majorité des cas.

Description du code :

1. Chargement et édition du framework

- Une zone de texte permet d'éditer ou coller un code Python définissant un objet `AFMatrix`.
- Un bouton "Lancer la génération" exécute ce code avec `exec(...)`, puis génère les points via `weightedHClassifierGenerator`
- Les messages de progression s'affichent en temps réel grâce à un callback intégré à Streamlit (`st.progress + st.text`).

2. Sélection des dimensions à visualiser

- Une fois les points calculés, l'utilisateur choisit 3 dimensions à afficher.
- Pour le cas $n > 3$ dimensions, l'interface génère des sliders dynamiques (un par dimension non visualisée).

3. Visualisation graphique

- En fonction des dimensions choisies et des valeurs des sliders, les points sont filtrés avec `filter_and_project`, selon la méthode d'hyper-slice décrite plus haut.
- Le convex hull est recalculé à chaque modification.
- Affichage en :
 - 1D : segment,
 - 2D : polygone convexe via `scipy.spatial.ConvexHull`,
 - 3D : affichage des faces via `plotly.graph_objects.Mesh3d`, avec stylisation.

4. Interface utilisateur

- L'affichage est divisé en colonnes pour une lecture claire.
- Un menu d'aide interactif (`st.expander`) explique les fonctionnalités.
- L'ensemble du layout est fluide, les boutons déclenchent des actions immédiates, sans reload de page.

Remarques suite aux tests

Une fois passée l'étape de génération de l'argumentation framework et du weighted H Classifier, la génération du convex hull est fluide. La génération du hull en 3D est ultra-rapide et bien adaptée à l'objectif du projet.

Lors de nos tests, nous avons observé une **potentielle non-linéarité** de la fonction reliant le convex hull aux dimensions. Lors de la modification de certains sliders, il arrive que le déplacement de 0.01 d'un slider génère un changement drastique de la forme du convex hull. Faute de pouvoir faire varier plus finement que 0.01 les sliders, et à cause de l'intervalle de tolérance, il est dur d'être sûr qu'il y a une vraie discontinuité, et pas juste une variation rapide de la forme. Mais si le phénomène de discontinuité était avéré, ce serait un résultat intéressant.

Cette discontinuité pourrait être causée par un phénomène de seuils de stabilité dans la sémantique choisie : lorsque le poids d'un des arguments converge au-delà d'un certain seuil critique, il force une convergence brutale d'autres poids vers un nouveau régime d'équilibre, tel que 0 ou 1 par exemple.

Si vous avez plus d'informations sur le sujet, nous serions curieux d'en savoir plus.

Quelles utilisations potentielles ? Lien avec l'admissibilité

Pour aller plus loin, la visualisation qu'offre le convex hull pourrait nous permettre de définir des espaces d'acceptabilité, associés à un argumentation framework et à une sémantique donnée. Si un point est situé à l'intérieur du convex hull, c'est qu'il existe au moins un ensemble de poids initiaux qui mènent à cet état stable. On pourrait donc le considérer comme "admissible".

Les formes du hull pourraient alors nous apporter d'autres informations :

- un hull large semblerait suggérer une grande variabilité des états d'acceptabilité en fonction des poids initiaux, donc un argumentation framework plus sensible aux poids
- un hull étroit indiquerait au contraire une forte stabilité : le système converge vers des états similaires quelle que soit le vecteur de poids initial

Parmi les autres idées d'applications potentielles :

- Comparaison de plusieurs sémantiques : qui est la plus restrictive / permissive ?
- Analyse des "sommets" du convex hull : états limites d'acceptabilité
- Visualisation graphique de l'admissibilité d'un point : est-il à l'intérieur ou à l'extérieur du convex hull ? A quelle distance de la limite d'acceptabilité se trouve-t-il ?